

KV Cache Sizing, Worked Out by Hand

WHAT ONE REQUEST ACTUALLY COSTS IN HBM — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It takes one concrete serving target — the *Nexus* fraud-detection assistant, a 70-billion-parameter Llama-3-class decoder with grouped-query attention (GQA-8) — and computes, byte by byte, how much *KV cache* memory a single 6200-token request consumes. Nothing is asserted without arithmetic. Every figure below is produced by a small reference script (Appendix A, `m04_t01_engine.py`) so the numbers are exact and reproducible. By the end you will be able to derive, on paper, why one request costs ~ 2.03 GB, why that number scales linearly with both tokens and layers, and why halving the KV dtype to FP8 is one of the highest-leverage moves in the whole serving stack.

The one idea. Model *weights* are static and shared across every request. The *KV cache* is per-request and grows with every token. At production concurrency the KV cache — not the weights — becomes the dominant memory constraint. This document makes that claim quantitative.

CONCEPTS COVERED, IN ORDER

what the KV cache stores; the per-layer formula $2n_{\text{kv}}d_{\text{head}}Tb$; the full per-request footprint over 80 layers; why GQA-8 saves $8\times$ versus MHA; the per-token budgeting rate; linear scaling in tokens and layers; the effect of FP8 KV; and why KV dominates HBM at concurrency.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system under study	2
2	Step 1 — What we store, per layer	2
3	Step 2 — The full per-request footprint	3
4	Step 3 — Why GQA-8 matters: the MHA counterfactual	3
5	Step 4 — The per-token rate you actually budget with	4
6	Step 5 — Linear scaling, in both directions	4
7	Step 6 — The FP8 KV lever	4
8	Step 7 — Why KV is the dominant constraint	5
9	The argument, at a glance	5

9.1 Equation cheat-sheet	5
Appendix A — Reference implementation	6

1 The system under study

We fix one concrete instance so every quantity is a number you can check.

Quantity	Symbol	Value here
KV heads (after GQA-8)	n_{kv}	8
Query heads (for the MHA comparison)	n_q	64
Head dimension	d_{head}	128
Decoder layers	N_{layers}	80
KV precision (baseline)	b	FP16 (2 bytes/element)
Context length	T	6200 tokens

We use decimal units for the big aggregate numbers: 1 GB = 10^9 B — the same convention vendors use for "141 GB of HBM." The one place we switch to *binary* units (1 KiB = 1024 B) is the per-token rate, because that is how serving engines quote block sizes. We flag it explicitly when we do.

Intuition

A decoder produces token n by attending to the keys and values of all previous tokens. Rather than recompute those K and V tensors every step, we *cache* them — that is the KV cache. It is the model's short-term memory of the conversation so far. The crucial fact: weights are loaded once and reused by every request, but the KV cache is *private to one request* and grows one token's worth of K and V at a time. Sizing it is the first step in every KV deployment spec.

2 Step 1 — What we store, per layer

At each layer, attention needs the keys and values for every token in context. For one token at one layer, K is a vector of length $n_{kv} \cdot d_{head}$ and V is the same size. So the bytes for the whole context at one layer are:

Per-request KV — one layer

$$kv_{layer} = \underbrace{2}_{\text{K and V}} \times n_{kv} \times d_{head} \times T \times b.$$

Plug in the numbers

$$kv_{layer} = 2 \times 8 \times 128 \times 6200 \times 2 = 2.5395 \times 10^7 \text{ B} = 25.4 \text{ MB}.$$

The factor of 2 at the front is not the dtype — it is "K *and* V," two separate tensors of identical shape. The dtype $b = 2$ appears separately at the end.

Mini-example — the operation in isolation

“Bytes from a shape” is just multiplication. One layer holds 25.4 MB of KV for a 6200-token request. A short 200-token request at the same layer holds $2 \cdot 8 \cdot 128 \cdot 200 \cdot 2 = 819$ KB — $31\times$ less. KV is strictly proportional to how many tokens you have seen.

3 Step 2 — The full per-request footprint

A forward pass runs all 80 layers, and each keeps its own KV. The total is just the per-layer figure times the layer count.

Per-request KV — full model

$$\text{kv}_{\text{total}} = \text{kv}_{\text{layer}} \times N_{\text{layers}} = (2 n_{\text{kv}} d_{\text{head}} T b) \times N_{\text{layers}}.$$

Plug in the numbers

$$\text{kv}_{\text{total}} = 25.4 \text{ MB} \times 80 = 2.0316 \times 10^9 \text{ B} = 2.03 \text{ GB per request. } \checkmark$$

The number that shocks every first-time engineer

One request. One model. A modest 6k context. 2.03 GB of HBM. And that is *with* GQA-8 already shrinking it $8\times$. The weights are loaded once and shared; this 2 GB is paid *again for every concurrent request*. KV memory, not weights, is what limits how many users one GPU can serve.

4 Step 3 — Why GQA-8 matters: the MHA counterfactual

Grouped-query attention shares one KV head across a group of query heads. Nexus has $n_q = 64$ query heads but only $n_{\text{kv}} = 8$ KV heads — a group size of 8. The KV cache scales with the number of KV heads, not query heads, so the saving is exactly the group size.

Same model, no GQA (full multi-head attention)

With 64 KV heads instead of 8:

$$\text{kv}_{\text{MHA}} = 2 \times 64 \times 128 \times 6200 \times 2 \times 80 = 1.625 \times 10^{10} \text{ B} = 16.25 \text{ GB per request.}$$

That is $64/8 = 8\times$ the GQA-8 cost.

Watch out

16.3 GB of KV *per request* would make a 70B model nearly unservable at any real concurrency — which is precisely why Llama-3 did not ship without GQA. When you read a model card, the ratio n_q/n_{kv} is a KV-memory multiplier hiding in plain sight. MHA ($n_{\text{kv}} = n_q$) is the expensive extreme; multi-query attention ($n_{\text{kv}} = 1$) is the cheap extreme; GQA sits in between.

5 Step 4 — The per-token rate you actually budget with

Capacity planning is easier in "bytes per token" than "bytes per request," because requests have different lengths but the per-token rate is fixed by the architecture. Drop T from the formula:

KV per token — the budgeting rate

$$\text{kv}_{\text{tok}} = 2 n_{\text{kv}} d_{\text{head}} b N_{\text{layers}}.$$

Plug in the numbers (binary KiB here)

$$\text{kv}_{\text{tok}} = 2 \times 8 \times 128 \times 2 \times 80 = 327680 \text{ B} = \frac{327680}{1024} = 320 \text{ KiB/token (FP16)}.$$

Sanity check against Step 2: $327680 \text{ B/token} \times 6200 \text{ tokens} = 2.03 \text{ GB}$. ✓

This single number — 320 KiB per token at FP16 — is the rate that drives block pool sizing and concurrency limits (Topic 5). Everything downstream multiplies it by a token count.

6 Step 5 — Linear scaling, in both directions

The formula is a product, so the footprint is *linear* in T and linear in N_{layers} . Doubling either doubles the KV. There are no economies of scale to hide behind.

Context T (tokens)	KV per request (FP16)		
512	0.168 GB	short chat turn	
2000	0.655 GB	typical RAG query	
6200	2.032 GB	Nexus full context ✓	
8192	2.684 GB	max context slot	

Intuition

Because KV is linear in tokens, the cost of a long context is unforgiving: a request that runs to the 8k limit reserves 2.68 GB for its *entire lifetime*, even while it is only emitting 180 output tokens. Naively reserving the maximum for every request is the fragmentation disaster that PagedAttention (Topic 2) exists to fix. But you cannot fix what you have not measured — and the measurement is this table.

7 Step 6 — The FP8 KV lever

The dtype b is a literal multiplier on every KV byte. Storing K and V in FP8 (1 byte) instead of FP16 (2 bytes) halves the entire cache.

Halve the dtype, halve the bytes

$$\text{kv}_{\text{total}}^{\text{FP8}} = 2 \times 8 \times 128 \times 6200 \times 1 \times 80 = 1.016 \times 10^9 \text{ B} = 1.02 \text{ GB per request},$$

$$\text{kv}_{\text{tok}}^{\text{FP8}} = 2 \times 8 \times 128 \times 1 \times 80 = 163840 \text{ B} = 160 \text{ KiB/token}.$$

FP8 KV: 160 KiB per token

Quantizing the KV cache to FP8 cuts the per-request footprint from 2.03 GB to 1.02 GB and the per-token rate from 320 to **160** KiB/token. That 160 KiB/token is the exact figure the block-pool sizing in Topic 5 uses to fit $\sim 649\text{k}$ tokens into a 99 GiB pool. KV quantization roughly *doubles* concurrency for a small, well-studied accuracy cost — which is why FP8 KV is now a default in production serving.

8 Step 7 — Why KV is the dominant constraint

Weights are paid once. KV is paid per concurrent request. Multiply the per-request footprint by realistic concurrency and the scale of the problem appears.

Concurrent requests	Aggregate KV (FP16)		
50	102 GB	already exceeds one H100's 80 GB	
200	406 GB	several GPUs of KV	
800	1625 GB	~ 20 H200s, just for KV ✓	

Watch out

No one actually pre-allocates 1.6 TB. The number above is the *naive* ceiling — what you would need if every concurrent request reserved its full context. The rest of Module 04 is a sequence of techniques (paging, prefix sharing, eviction, offloading, disaggregation) that close the gap between this naive ceiling and what a GPU can actually hold. Topic 1's job is only to establish how big the number is, and where it comes from.

9 The argument, at a glance

#	Step	Result
1	Per layer	$\text{kv}_{\text{layer}} = 2 n_{\text{kv}} d_{\text{head}} T b = 25.4 \text{ MB}$
2	Per request	$\times N_{\text{layers}} = 2.03 \text{ GB}$
3	GQA vs MHA	$\text{MHA} = 16.25 \text{ GB} = 8 \times \text{GQA-8}$
4	Per token	$2 n_{\text{kv}} d_{\text{head}} b N_{\text{layers}} = 320 \text{ KiB/token}$
5	Scaling	linear in T and in N_{layers}
6	FP8 KV	1.02 GB/req, 160 KiB/token (half)
7	At scale	800 reqs $\Rightarrow$ 1.6 TB naive KV

9.1 Equation cheat-sheet

Per-layer KV	$\text{kv}_{\text{layer}} = 2 n_{\text{kv}} d_{\text{head}} T b$
Per-request KV	$\text{kv}_{\text{total}} = \text{kv}_{\text{layer}} \times N_{\text{layers}}$
Per-token rate	$\text{kv}_{\text{tok}} = 2 n_{\text{kv}} d_{\text{head}} b N_{\text{layers}}$
GQA saving	n_q/n_{kv} (MHA $\rightarrow$ GQA multiplier)
FP8 lever	$b : 2 \rightarrow 1 \Rightarrow$ footprint halved
Aggregate	$\text{KV}_{\text{total}} = C \times \text{kv}_{\text{total}}$ (C = concurrency)

Appendix A — Reference implementation

Every number above is produced by `m04_t01_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change n_{kv} , T , or the dtype to explore other models and precisions.

```

1  """
2  Reference implementation for: Module 04 - Topic 1
3  "KV Cache Sizing - what one request actually costs in HBM."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Convention: decimal for HBM/bandwidth (1 GB = 1e9 B), but per-token KV is
9  quoted in BINARY KiB/MiB the way serving engines report block sizes
10 (1 KiB = 1024 B). Both conventions are stated where they are used.
11 """
12
13 # -----
14 # 0. The system under study: Nexus, a Llama-3-70B-class decoder, GQA-8.
15 # -----
16 n_kv_heads = 8          # KV heads after grouped-query attention (GQA-8)
17 n_q_heads   = 64        # query heads (for the MHA comparison)
18 head_dim    = 128       # per-head width
19 n_layers    = 80        # decoder layers
20 dtype_fp16  = 2         # bytes per KV element at FP16
21 dtype_fp8   = 1         # bytes per KV element at FP8
22 T           = 6200      # Nexus context length (tokens)
23
24 # -----
25 # 1. Per-layer, per-request KV bytes: 2 (K and V) * n_kv * d_head * T * dtype
26 # -----
27 def kv_layer_bytes(n_kv, T, dtype):
28     return 2 * n_kv * head_dim * T * dtype
29
30 kv_layer = kv_layer_bytes(n_kv_heads, T, dtype_fp16)
31 kv_total = kv_layer * n_layers
32 print("== 1. Per-request KV (FP16, GQA-8) ==")
33 print(f"kv_per_layer = 2 x {n_kv_heads} x {head_dim} x {T} x {dtype_fp16} "
34       f"= {kv_layer:.4e} B = {kv_layer/1e6:.1f} MB")
35 print(f"kv_total      = {kv_layer/1e6:.1f} MB x {n_layers} = "
36       f"{kv_total:.4e} B = {kv_total/1e9:.3f} GB per request")
37
38 # -----
39 # 2. The MHA counterfactual: same model with 64 KV heads (no GQA).
40 # -----
41 kv_mha = kv_layer_bytes(n_q_heads, T, dtype_fp16) * n_layers
42 print("\n== 2. Counterfactual: MHA (64 KV heads, no GQA) ==")
43 print(f"kv_total_MHA = {kv_mha/1e9:.2f} GB per request "
44       f"f"= {n_q_heads//n_kv_heads}x the GQA-8 cost)")
45
46 # -----
47 # 3. The per-token rate (the number you actually budget with).
48 #   kv_bytes_per_token = 2 * n_kv * d_head * dtype * n_layers
49 # -----
50 def kv_per_token(n_kv, dtype):
51     return 2 * n_kv * head_dim * dtype * n_layers
52
53 pt_fp16 = kv_per_token(n_kv_heads, dtype_fp16)
54 print("\n== 3. KV per token (the budgeting rate) ==")
55 print(f"kv_per_token (FP16) = 2 x {n_kv_heads} x {head_dim} x {dtype_fp16} x {n_layers}"
56       f"= {pt_fp16} B = {pt_fp16/1024:.0f} KiB/token")
57 print(f"check: {pt_fp16} B/token x {T} tokens = {pt_fp16*T/1e9:.3f} GB (matches step 1)")

```

```

58
59 # -----
60 # 4. Scaling: KV is linear in tokens and linear in layers.
61 # -----
62 print("\n== 4. Linear scaling with context length ==")
63 for t in [512, 2000, 6200, 8192]:
64     g = kv_per_token(n_kv_heads, dtype_fp16) * t / 1e9
65     print(f"  T={t:5d} tokens -> {g:.3f} GB/request")
66
67 # -----
68 # 5. FP8 KV: halve the dtype, halve the bytes.
69 # -----
70 kv_fp8 = kv_layer_bytes(n_kv_heads, T, dtype_fp8) * n_layers
71 pt_fp8 = kv_per_token(n_kv_heads, dtype_fp8)
72 print("\n== 5. FP8 KV (1 byte/element) ==")
73 print(f"kv_total_FP8 = {kv_fp8/1e9:.3f} GB per request (half of FP16 {kv_total/1e9:.3f} GB)")
74 print(f"kv_per_token_FP8 = {pt_fp8} B = {pt_fp8/1024:.0f} KiB/token")
75
76 # -----
77 # 6. Why KV dominates: aggregate over concurrent requests.
78 # -----
79 print("\n== 6. Aggregate KV at concurrency ==")
80 for conc in [50, 200, 800]:
81     print(f"  {conc:4d} concurrent x {kv_total/1e9:.2f} GB = {conc*kv_total/1e9:.0f} GB of KV")

```

— END OF DOCUMENT —